

Python Level 1 — Coding Challenge Prep Sheet

20 core problems (with working code) + a bonus list/tuple/set/dict quick-reference — built from your syllabus: fundamentals, functions, args/kwargs, modules, files & OOP.

Q1. Prime Number

Loops · Math

Check whether a number is prime by testing divisibility up to its square root.

Example: Input: 29 → Output: 29 is prime

```
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True
```

```
n = 29
print(f"{n} is prime" if is_prime(n) else f"{n} is not prime")
```

Q2. Armstrong Number

Strings · Math

A number equal to the sum of its own digits each raised to the power of the digit count.

Example: Input: 153 → Output: 153 is Armstrong ($1^3 + 5^3 + 3^3 = 153$)

```
def is_armstrong(n):
    digits = str(n)
    power = len(digits)
    total = sum(int(d) ** power for d in digits)
    return total == n
```

```
n = 153
print(f"{n} is Armstrong" if is_armstrong(n) else f"{n} is not Armstrong")
```

Q3. Strong Number

Strings · Math

A number equal to the sum of the factorials of its digits.

Example: Input: 145 → Output: 145 is Strong ($1! + 4! + 5! = 145$)

```
import math

def is_strong(n):
    return sum(math.factorial(int(d)) for d in str(n)) == n
```

```
n = 145
print(f"{n} is Strong" if is_strong(n) else f"{n} is not Strong")
```

Q4. Anagram Check

Strings

Two strings are anagrams if they contain the same characters (ignoring case & spaces) in any order.

Example: Input: "listen", "silent" → Output: True

```
def is_anagram(s1, s2):
    s1 = s1.replace(" ", "").lower()
    s2 = s2.replace(" ", "").lower()
    return sorted(s1) == sorted(s2)

print(is_anagram("listen", "silent"))    # True
print(is_anagram("hello", "world"))      # False
```

Q5. Character Frequency

Strings · Dict

Count how many times each character appears in a string.

Example: Input: "hello" → Output: {'h': 1, 'e': 1, 'l': 2, 'o': 1}

```
def char_freq(s):
    freq = {}
    for ch in s:
        freq[ch] = freq.get(ch, 0) + 1
    return freq

print(char_freq("hello"))

# One-liner using Counter:
# from collections import Counter
# print(dict(Counter("hello")))
```

Q6. Remove Duplicate Characters

Strings · Set

Remove repeated characters while preserving the first-seen order.

Example: Input: "programming" → Output: "progamin"

```
def remove_dup_chars(s):
    seen = set()
    result = ""
    for ch in s:
        if ch not in seen:
            seen.add(ch)
            result += ch
    return result

print(remove_dup_chars("programming"))
```

Q7. String Compression

Strings

Compress consecutive repeated characters into character + count form.

Example: Input: "aaabbbc" → Output: "a3b2c1"

```
def compress(s):
    result = ""
    count = 1
    for i in range(1, len(s) + 1):
        if i < len(s) and s[i] == s[i - 1]:
```

```

        count += 1
    else:
        result += s[i - 1] + str(count)
        count = 1
    return result

print(compress("aaabbc"))    # a3b2c1

```

Q8. Second Largest (without sort)

Lists

Find the second largest number in one pass, without using sort().

Example: Input: [10, 5, 20, 20, 8] → Output: 10

```

def second_largest(nums):
    first = second = float("-inf")
    for n in nums:
        if n > first:
            second = first
            first = n
        elif first > n > second:
            second = n
    return second

print(second_largest([10, 5, 20, 20, 8]))    # 10

```

Q9. Remove Duplicates from List

Lists · Set

Remove duplicate elements from a list while preserving original order.

Example: Input: [1, 2, 2, 3, 1, 4] → Output: [1, 2, 3, 4]

```

def remove_dup_list(lst):
    seen = set()
    result = []
    for item in lst:
        if item not in seen:
            seen.add(item)
            result.append(item)
    return result

print(remove_dup_list([1, 2, 2, 3, 1, 4]))

```

Q10. Bubble Sort

Lists · Sorting

Sort a list by repeatedly swapping adjacent out-of-order elements.

Example: Input: [5, 2, 9, 1, 5, 6] → Output: [1, 2, 5, 5, 6, 9]

```

def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

print(bubble_sort([5, 2, 9, 1, 5, 6]))

```

Q11. Binary Search

Lists · Searching

Find a target's index in a sorted list by halving the search space each step.

Example: Input: [1,3,5,7,9,11], target=7 → Output: index 3

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

print(binary_search([1, 3, 5, 7, 9, 11], 7))    # 3
```

Q12. Matrix Addition

Lists · Nested

Add two matrices (2D lists) element-wise using nested list comprehension.

Example: Input: [[1,2],[3,4]] + [[5,6],[7,8]] → Output: [[6,8],[10,12]]

```
def matrix_add(a, b):
    return [[a[i][j] + b[i][j] for j in range(len(a[0]))]
            for i in range(len(a))]

A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]
print(matrix_add(A, B))    # [[6, 8], [10, 12]]
```

Q13. Matrix Transpose

Lists · Nested

Flip a matrix over its diagonal — rows become columns.

Example: Input: [[1,2,3],[4,5,6]] → Output: [[1,4],[2,5],[3,6]]

```
def transpose(m):
    return [[m[r][c] for r in range(len(m))] for c in range(len(m[0]))]

M = [[1, 2, 3], [4, 5, 6]]
print(transpose(M))    # [[1, 4], [2, 5], [3, 6]]

# Shortcut: list(map(list, zip(*M)))
```

Q14. Word Frequency

Strings · Dict

Count how many times each word occurs in a sentence.

Example: Input: "the cat sat on the mat the cat ran" → Output: {'the': 3, 'cat': 2, ...}

```
def word_freq(text):
```

```

words = text.lower().split()
freq = {}
for w in words:
    freq[w] = freq.get(w, 0) + 1
return freq

```

```
print(word_freq("the cat sat on the mat the cat ran"))
```

Q15. CSV Student Marks (average + topper)

File I/O · csv

Read student records from a CSV and compute the class average and topper.

Example: Input: CSV rows name,marks → Output: Average + Topper name & marks

```

import csv
from io import StringIO

data = """name,marks
Asha,88
Ravi,95
Meena,79"""

reader = csv.DictReader(StringIO(data))          # use open("file.csv") in real use
students = [(row["name"], int(row["marks"])) for row in reader]

total = sum(marks for _, marks in students)
average = total / len(students)
topper = max(students, key=lambda x: x[1])

print(f"Average: {average:.2f}")
print(f"Topper: {topper[0]} ({topper[1]} marks)")

```

Q16. Calculator using Functions

Functions

Build a calculator that dispatches to the right operation function based on input.

Example: Input: '+', 5, 3 → Output: 8

```

def add(a, b): return a + b
def sub(a, b): return a - b
def mul(a, b): return a * b
def div(a, b): return a / b if b != 0 else "Error: division by zero"

def calculate(op, a, b):
    ops = {"+": add, "-": sub, "*": mul, "/": div}
    return ops.get(op, lambda a, b: "Invalid operator")(a, b)

print(calculate("+", 5, 3))    # 8
print(calculate("/", 5, 0))    # Error: division by zero

```

Q17. Bank Account Class

OOP · Classes

Model a bank account with deposit/withdraw methods and balance validation.

Example: deposit(500) → Balance: 1500 | withdraw(200) → Balance: 1300

```
class BankAccount:
```

```

def __init__(self, owner, balance=0):
    self.owner = owner
    self.balance = balance

def deposit(self, amount):
    self.balance += amount
    print(f"Deposited {amount}. Balance: {self.balance}")

def withdraw(self, amount):
    if amount > self.balance:
        print("Insufficient funds")
    else:
        self.balance -= amount
        print(f"Withdrew {amount}. Balance: {self.balance}")

acc = BankAccount("Kumar", 1000)
acc.deposit(500)
acc.withdraw(200)

```

Q18. Student Class

OOP · Classes

Model a student with a list of marks; compute average and pass/fail status.

Example: marks=[78,65,90] → average 77.67 → Pass

```

class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks          # list of subject marks

    def average(self):
        return sum(self.marks) / len(self.marks)

    def is_pass(self, pass_mark=40):
        return all(m >= pass_mark for m in self.marks)

s = Student("Neha", [78, 65, 90])
print(f"{s.name}'s average: {s.average():.2f}")
print("Pass" if s.is_pass() else "Fail")

```

Q19. Multiple Exception Handling

Exceptions

Handle several distinct error types in one function using multiple except blocks.

Example: safe_divide(10, 0) → 'Error: cannot divide by zero'

```

def safe_divide(a, b):
    try:
        result = a / b
    except ZeroDivisionError:
        return "Error: cannot divide by zero"
    except TypeError:
        return "Error: invalid input types"
    else:
        return result
    finally:
        print("Division attempt complete")

print(safe_divide(10, 2))

```

```
print(safe_divide(10, 0))
print(safe_divide(10, "a"))
```

Q20. File Word Count

File I/O

Count the number of words in a text file, handling the missing-file case.

Example: sample.txt with 42 words → Output: 42

```
def count_words(filepath):
    try:
        with open(filepath, "r") as f:
            text = f.read()
            return len(text.split())
    except FileNotFoundError:
        return "File not found"

# Example usage:
# print(count_words("sample.txt"))
```

Bonus: List / Tuple / Set / Dict Quick Patterns

These patterns get folded into 'easy' assessments constantly — comfortable recall here saves you time on everything above.

List Comprehension — filter + transform

List

```
nums = [1, 2, 3, 4, 5, 6, 7, 8]
evens_squared = [n ** 2 for n in nums if n % 2 == 0]
print(evens_squared)    # [4, 16, 36, 64]
```

Tuple Unpacking

Tuple

```
point = (3, 7)
x, y = point
print(f"x={x}, y={y}")

# Swapping without a temp variable
a, b = 1, 2
a, b = b, a
print(a, b)    # 2 1
```

Set Operations

Set

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a | b)    # union -> {1,2,3,4,5,6}
print(a & b)    # intersection -> {3,4}
print(a - b)    # difference -> {1,2}
print(a ^ b)    # symmetric difference -> {1,2,5,6}
```

Dictionary Comprehension + Sort by Value

Dict

```
prices = {"tea": 20, "coffee": 35, "juice": 15}
with_gst = {item: round(p * 1.05, 2) for item, p in prices.items()}
print(with_gst)

# Sort dict by value, descending
ranked = dict(sorted(prices.items(), key=lambda x: x[1], reverse=True))
print(ranked)
```

zip() — Combine Lists Pairwise

List

```
names = ["Asha", "Ravi", "Meena"]
scores = [88, 95, 79]

combined = dict(zip(names, scores))
print(combined)    # {'Asha': 88, 'Ravi': 95, 'Meena': 79}
for name, score in zip(names, scores):
```



```
print(f"{name}: {score}")
```

enumerate() — Index + Value Together

List

```
fruits = ["apple", "banana", "cherry"]
for i, fruit in enumerate(fruits, start=1):
    print(f"{i}. {fruit}")
```

Flatten a Nested List

List

```
nested = [[1, 2], [3, 4], [5, 6]]
flat = [num for row in nested for num in row]
print(flat)    # [1, 2, 3, 4, 5, 6]
```

List vs Tuple vs Set vs Dict — quick recall

Concept

```
# list  -> ordered, mutable, allows duplicates    [1, 2, 2]
# tuple -> ordered, immutable, allows duplicates (1, 2, 2)
# set   -> unordered, mutable, no duplicates     {1, 2}
# dict  -> key-value pairs, mutable, unique keys {"a": 1}

# Rule of thumb:
# Need to change it later?      -> list
# Must never change (locked)?   -> tuple
# Only care about uniqueness?   -> set
# Need fast lookup by a label?   -> dict
```

How to use this sheet: Don't just read the code — cover it, re-derive the logic from the description + example, then check yourself against this. That's what actually builds assessment-speed recall.